# EXPERIMENT NO. 04 (Software) – Age Scheduling in Network




*Name: Dibyanshu Shekhar Dey*
*Roll Number: 25EC64R23*

**Design and Simulation Lab: Communication Networks**
Department of Electronics & Electrical Communication Engineering
Indian Institute of Technology Kharagpur

**AIM:**
**Primary Tasks:**

- Model a Network as a Queue-Server in matlab where there a Source generates and sends packets and the server provides service to these packets as a random process (exponentially distributed).
- Find Average Age at the monitor under the queue Scheduling policies:
  - FCFS
  - LCFS
  - Random Scheduling

  and plot the Age vs Load plot for each scheduling policy. and comment about your observations in each plots and their comparison.

**Secondary Tasks:**

- Do Queue scheduling for Round Robin.
- Generate a Sawtooth Age plot (under FCFS).

---

**INTRODUCTION:**
Real-time status updates play a critical role in enabling modern applications such as intelligent transportation systems, smart homes, and environmental monitoring. These systems rely on time-stamped messages generated by sensors or devices that reflect the current state of a physical process. For instance, temperature sensors in forests can provide updates that help predict fires, while cars may exchange velocity and acceleration data to support safe driving decisions. In all these scenarios, the freshness of the information—referred to as the *age of information*—is vital.

The *age of information* at a monitor is defined as the time elapsed since the generation of the latest received packet or status update. Maintaining a low average age is essential for the monitor to make timely decisions. However, ensuring minimal age is a complex challenge due to the trade-offs involved. Simply increasing the update rate can congest the system, causing delays and defeating the purpose of frequent updates. On the other hand, reducing the update rate too much leads to stale information at the monitor.

This experiment explores a simple fundamental question: **Effect of Queue Scheduling Policy on Age at Monitor**. We consider queue-based communication systems where packets experience non-zero and random arrival (from a source) and service times (by the server) reaching to a monitor. Our focus is using queue scheduling with different policies with arrival and service times that may be independent and identically distributed (i.i.d.), as in M/M/1/B queueing systems, and compare these policies with Age of Information as a metric. We will generate Age plots under different scheduling policies with different load on the server (i.e., $\rho = \frac{\lambda}{\mu}$). Insights from these experiments will help you understand the importance of freshness of information under given constraints, which is crucial for designing efficient real-time monitoring systems in practical, delay-sensitive environments.

**THEORY:**
**Age of Information:**

The *Age of Information (AoI)* is a metric that helps in quantifying the "freshness" of information at a monitor/destination in a communication system [1]. If we consider a source that generates status updates which are transmitted through a network to a destination monitor, the monitor then uses these updates to maintain knowledge of the state of a physical process.

Assume $u(t)$ denote the generation time of the most recently received update packet at time $t$. Then Age of Information at time $t$ is defined as

$$\Delta(t) = t - u(t).$$

This definition implies that the age of information measures how old the newest received information is [1]. Immediately after a packet is delivered to the monitor, the age drops to the system time of that packet, and then increases linearly with time until the next update is received.
**Age Evolution Process** Let update packets be generated at times $t_1, t_2, \ldots$ and delivered to the destination monitor at times $t'_1, t'_2, \ldots$. Due to network delays, packets arrive at the monitor later than their generation times [1], i.e., $t'_i \geq t_i$.
When the $j$-th packet is delivered at time $t'_j$, the AoI is reset to

$$\Delta(t'_j) = t'_j - t_j$$

which represents the system time of the packet, consisting of queueing delay and service time [1]. Between successive packet deliveries, no new information arrives at the monitor, and therefore the age increases linearly with slope equal to one [1].

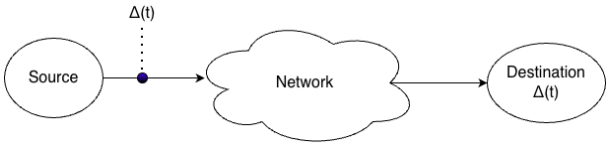



Figure 1: Network block diagram for analysing AoI

Figure 1 illustrates the Network diagram that shows how the AoI is being processed in the system.

- $\Delta_1(t)$ : Age observed at the input of the network (Monitor 1).
- $\Delta(t)$ : Age observed at the destination monitor.

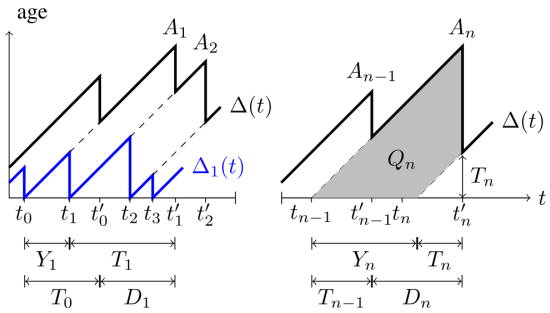



Figure 2: Network block diagram for analysing AoI [1]

As per Figure 2, at the network input, updates are observed immediately when they are generated. Therefore, the age $\Delta_1(t)$ resets to zero at each generation time $t_i$. Between successive update generations, the age increases linearly.

At the destination monitor, updates are received only after passing through the network. Hence, the age $\Delta(t)$ resets at the delivery times $t_i'$ instead of the generation times $t_i$. As a result, the age reset value is not zero, but equal to $t_i'$ - $t_i$ which is the system delay in update. Consequently, the AoI process on the destination monitor forms a characteristic *sawtooth pattern*. Each sawtooth segment corresponds to the period between two consecutive update deliveries [1].

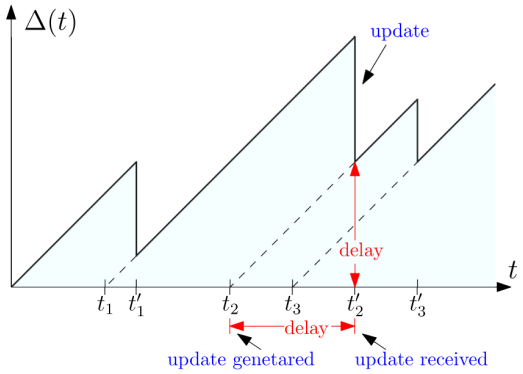



Figure 3: A Simplified diagram for analyzing AoI. The sharp decrease mean that the packet has arrived at the destination.[2]

**Time Average Age**: To evaluate the overall freshness of information, the time average age over an interval $(0, T)$ is defined as

$$\langle \Delta \rangle_T = \frac{1}{T} \int_0^T \Delta(t) dt.$$

Graphically, this means total area under the AoI curve that is divided by the observation time [1], [3].

**Graphical Evaluation of Average Age:**Let

$$Y_n = t_n' - t_{n-1}'$$

denote the inter-delivery time between two consecutive updates. Let

$$T_n = t_n' - t_n$$

denote the system time of the $n$-th update [1].
Between two deliveries, the AoI curve forms a trapezoidal region whose area is denoted by $Q_n$. From the geometry of the figure, the area of this region is [1], [3]

$$Q_n = \frac{1}{2}(T_n + Y_n)^2 - \frac{1}{2}T_n^2.$$

Simplifying,

$$Q_n = Y_n T_n + \frac{Y_n^2}{2}$$

The average AoI over time interval $T$ is therefore

$$\Delta = \frac{1}{T} \sum_n Q_n$$

**FCFS - M/M/1** - The experiment has to consider an FCFS $M/M/1$ queuing system with arrival rate $\lambda$ and service rate $\mu$. Update packets are generated according to a Poisson process with rate $\lambda$, resulting in inter arrival times $Y_i$ that are independent and identically distributed exponential random variables with mean $E[Y] = 1/\lambda$. Similarly the service times are i.i.d. exponential random variables with mean $1/\mu$. The objective is to evaluate the Age of Information (AoI) for this system and determine the server utilization $\rho = \lambda/\mu$ that minimizes the average age $\Delta$ [3].

**RESULTS:**

Results when $\lambda$ varies from 1 to 9, with buffer size as 5 and total simulation time takes as 10.

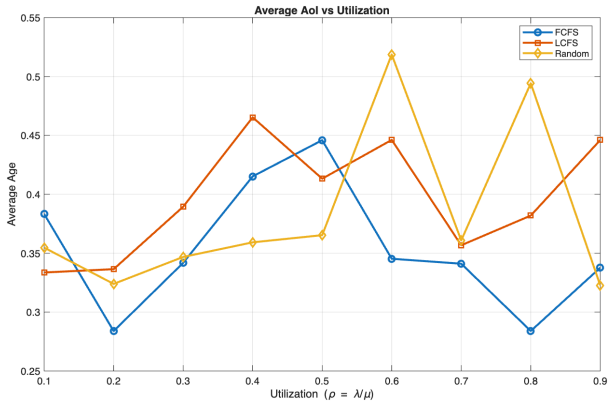



Figure 4: AoI vs Utilization for $\lambda$ varying from 1 to 9 and buffer size being 10.

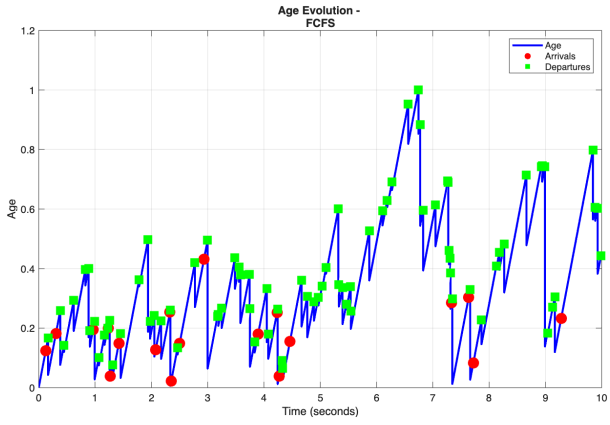



Figure 5: FCFS Sawtooth for $\lambda$ varying from 1 to 9 and buffer size being 10.

```
"Average age for FCFS..."    "0.33768"

"Average age for LCFS..."    "0.44607"

"Average age for Rand..."    "0.32267"
```

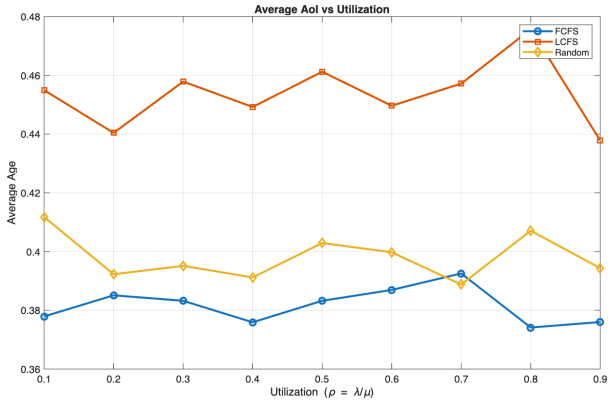



Figure 6: AoI vs Utilization for $\lambda$ varying from 1 to 9 and buffer size being 10.

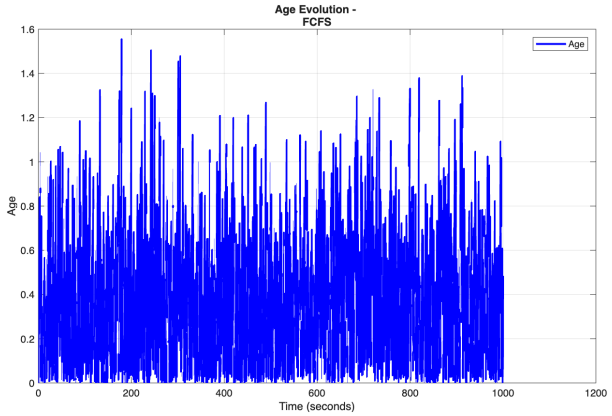



Figure 7: FCFS Sawtooth for $\lambda$ varying from 1 to 9 and buffer size being 10.

Average age results when $\lambda$ varies from 1 to 9, with buffer size as 5 and total simulation time takes as 1000.

```
"Average age for FCFS..."    "0.37601"

"Average age for LCFS..."    "0.43778"

"Average age for Rand..."    "0.39432"
```

**DISCUSSION:**
From the expression

$$Q_n = Y_n T_n + \frac{Y_n^2}{2},$$

two components contribute to the AoI:

- $Y_n T_n$ : contribution due to system delay of the packet.
- $\frac{Y_n^2}{2}$ : contribution due to the time gap between consecutive packet deliveries.

The results for the utilization factor is not as smooth as expected both in the paper as well as the presentation by the TA. The results follow a more jig jag pattern.
Average age results when $\lambda$ varies from 1 to 9, with buffer size as 5 and total simulation time takes as 10. However as the simulation time increases, it follows a smooth pattern for a fixed buffer size, but it is still not as smooth as the pattern in the papers and the presentation.

The distribution of the plots however is along the average age of methods that are being used. Once can infer that increase the value of lambda and simulation time for large values and for a fixed buffer size, the plot will more or less try to converge to the average age calculated.

**ACKNOWLEDGEMENT:**
I would like to share my thanks to Prof. Amitalok J Budkuley for explaining the context of the algorithms. I would like to extend my thanks to TA Anuj Meena for explaining the fundamentals of the experiment with great presentation and simulations, and helping throughout along with the problem description. I would like to thank my colleagues Mayank Singh, Saptarshi Bhattacharjee and Soumava Biswas for collaborating in lab throughout the experiment.

# References

[1] R. D. Yates, Y. Sun, D. R. Brown, S. K. Kaul, E. Modiano, and S. Ulukus, "Age of information: An introduction and survey," *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 5, pp. 1183–1210, 2021. DOI: 10.1109/JSAC.2021.3065072.

[2] P. Kaswan, P. Mitra, A. Srivastava, and S. Ulukus, *Age of information in gossip networks: A friendly introduction and literature survey*, 2023. arXiv: 2312.16163 [cs.IT]. [Online]. Available: https://arxiv.org/abs/2312.16163.

[3] S. Kaul, R. Yates, and M. Gruteser, "Real-time status: How often should one update?" In *2012 Proceedings IEEE INFOCOM*, 2012, pp. 2731–2735. DOI: 10.1109/INFCOM.2012.6195689.

**APPENDIX (CODE):**

**Main Script for the AoI Taks**

```
clear;
clc;
close all;

lambda_arr = 1:5; % various lambda values

avg_age_fcfs = []; % Initialize average age for FCFS
avg_age_lcfs = []; % Initialize average age for LCFS
avg_age_rand = []; % Initialize average age for Random scheduling

util_arr = []; % utilizxation array

%parameters
buff_size = 5;
tot_sim_time = 10;
mu = 10; % service rate
ser_mean = 1 / mu; % service mean

```

```
for lambda = lambda_arr
    arr_mean = 1 / lambda; % arrival mean
    rho = lambda / mu;
    util_arr = [util_arr rho]; % add rho into utilization array

    % FCFS
    [A_fcfs, D_fcfs] = queue_sim(tot_sim_time, buff_size, arr_mean,
        ser_mean, "FCFS"); % calling queue simulaton
    avg_age_fcfs = [avg_age_fcfs compute_aoi(A_fcfs, D_fcfs, tot_sim_time)
        ]; % calling average age function

    % LCFS
    [A_lcfs, D_lcfs] = queue_sim(tot_sim_time, buff_size, arr_mean,
        ser_mean, "LCFS"); % calling queue simulaton
    avg_age_lcfs = [avg_age_lcfs compute_aoi(A_lcfs, D_lcfs, tot_sim_time)
        ]; % calling average age function

    % Random Scheduling
    [A_rand, D_rand] = queue_sim(tot_sim_time, buff_size, arr_mean,
        ser_mean, "Random"); % calling queue simulaton
    avg_age_rand = [avg_age_rand, compute_aoi(A_rand, D_rand, tot_sim_time)
        ]; % calling average age function

end

disp(["Average age for FCFS Scheduling - " compute_aoi(A_fcfs, D_fcfs,
    tot_sim_time)]);
disp(["Average age for LCFS Scheduling - " compute_aoi(A_lcfs, D_lcfs,
    tot_sim_time)]);
disp(["Average age for Random Scheduling - " compute_aoi(A_rand, D_rand,
    tot_sim_time)]);

figure;
plot(util_arr,avg_age_fcfs,'-o','LineWidth',2);
hold on;
plot(util_arr,avg_age_lcfs,'-s','LineWidth',2);
plot(util_arr,avg_age_rand,'-d','LineWidth',2);

xlabel('Utilization␣(\rho␣=␣\lambda/\mu)')
ylabel('Average␣Age')
title('Average␣AoI␣vs␣Utilization')
legend('FCFS','LCFS','Random')
grid on

% plot of the Sawtooth plot
figure;
plot_sawtooth(A_fcfs, D_fcfs, "FCFS");
plot_sawtooth(A_lcfs, D_lcfs, "LCFS");
plot_sawtooth(A_rand, D_rand, "Random Scheduling");
hold on;
```

### Script for the AoI Queue Simulation

```
function [A_serv, D_serv] = queue_sim(tot_sim_time, buffer_size, arr_mean,
    ser_mean, method)

buffer = 0;
queue_arr = [];

```

```
next_arr = exprnd(arr_mean); % arrival rate follows exponenetial Rv, choose a value
next_ser = Inf;              % service time is infinity
t = 0;                       % tie instance initialised

A_serv = []; % array empty to check total number of packets arriving
D_serv = []; % array empty to check total number of packets departing

% check that time is less than total simulation time
while t < tot_sim_time
    if next_arr < next_ser % if arrival time is less than service time
        t = next_arr; % put the arrival time in t variable

        % the buffer capacity at current moment is less than buffer
        % capacity
        if buffer < buffer_size
            buffer = buffer + 1;
            queue_arr = [queue_arr t]; % append time in array (queue)

            if buffer == 1
                next_ser = t + exprnd(ser_mean); % total service = arr time + exp rv from service time
            end
        end
        next_arr = t + exprnd(ser_mean); % next arrival = arr time + exp rv from service time
    else
        t = next_ser; % service completion
        if buffer > 0 % buffer is already occupied
            if method == "FCFS"
                idx = 1;

            elseif method == "LCFS"
                idx = length(queue_arr);

            elseif method == "Random"
                idx = randi(length(queue_arr));
            end

        end
        arrival_time = queue_arr(idx); % take first index of arrival queue
        queue_arr(idx) = []; % remove that element from array

        buffer = buffer - 1; % decrease buffer size

        A_serv = [A_serv arrival_time]; % append the array for arrival time
        D_serv = [D_serv t]; % append the array for service time
    end

    if buffer > 0
        next_ser = t + exprnd(ser_mean); % update the service time if buffer > 0 else keep service time infinity
    else
        next_ser = Inf;
    end
end
end
```

### Script to calculate AoI

```
function avg_age = compute_aoi(A_serv, D_serv, T)

N = length(D_serv); % total Departure served
avg_age = 0;

for i = 2:N
    % avg_age = avg_age + (A_serv(i) - D_serv(i));
    Y = D_serv(i) - D_serv(i-1); % inter departure time
    T_sys = D_serv(i-1) - A_serv(i-1); % system time for packet i -1
    avg_age = avg_age + (Y^2)/2 + T_sys*Y; % area under curve, trapezoid
       arae

end
avg_age = avg_age / T;
end
```

### Plotting Sawtooth Calculation

```
function plot_sawtooth(A,D, method)

t_plot = [];
age_plot = [];

arrival_age = [];
departure_age = [];

current_age = 0;
last_dep = 0;

for i = 1:length(D)

    % age growth between two departures
    t = linspace(last_dep, D(i), 100);
    age = current_age + (t - last_dep);

    t_plot = [t_plot t];
    age_plot = [age_plot age];

    % age at arrival instant
    if A(i) >= last_dep && A(i) <= D(i)
        arr_age = current_age + (A(i) - last_dep);
        arrival_age = [arrival_age arr_age];
    else
        arrival_age = [arrival_age NaN];
    end

    % Age just before packet reception by system
    dep_age = current_age + (D(i) - last_dep);
    departure_age = [departure_age dep_age];

    % Age reset after reception by system
    current_age = D(i) - A(i);
    last_dep = D(i);

end

% plotting the functions
```

```
figure
plot(t_plot, age_plot, 'b', 'LineWidth', 2)
hold on

% plotting arrivals
scatter(A, arrival_age, 120, 'r', 'filled')

% plotting departures
scatter(D, departure_age, 120, 'g', 's', 'filled')

xlabel('Time (seconds)')
ylabel('Age')
title(['Age Evolution - ' method])
legend('Age','Arrivals','Departures')

grid on

end
```